



PROGRAMME DE FORMATION

# LLM et agents d'IA

Construire, sécuriser, évaluer et déployer des applications sur des grands modèles de langage

---

**27** HEURES

**POUR** Développeurs qui veulent construire des applications avec des grands modèles de langage



**Mohamed Aziz BEN HAH**

[mohamedazizbenhaha.netlify.app](https://mohamedazizbenhaha.netlify.app)

## Présentation

Ce programme apprend aux développeurs à construire des applications sur des grands modèles de langage et à les mettre en production de façon responsable. Les participants découvrent comment Transformers et LLM sont entraînés et où sont leurs limites, écrivent des prompts efficaces et des sorties structurées, utilisent des API hébergées et des modèles ouverts, construisent de la génération augmentée par la recherche (RAG) sur leurs propres documents, et donnent des outils aux modèles par l'appel de fonctions, les frameworks d'agents et le Model Context Protocol. Ils protègent ensuite l'application contre l'injection de prompt, l'évaluent avec des jeux de test et des LLM juges, suivent son coût et la servent comme API. Le programme s'achève par un projet final : une application d'agent construite, évaluée et présentée par chaque équipe.

## Vue d'ensemble

| #     | PHASE                       | THÈMES                                                                                      | HEURES |
|-------|-----------------------------|---------------------------------------------------------------------------------------------|--------|
| 01    | Fondamentaux des LLM        | Transformers et entraînement, prompts, sortie structurée, API et modèles ouverts            | 6 h    |
| 02    | RAG et agents               | Embeddings, bases vectorielles, recherche et reclassement, outils, frameworks d'agents      | 6 h    |
| 03    | MCP, sécurité et évaluation | Model Context Protocol, injection de prompt et garde-fous, évaluation, traçage, déploiement | 6 h    |
| 04    | Projet final                | Une application d'agent, évaluée et déployée                                                | 6 h    |
|       | Réserve                     | Rattrapage, révision, TP supplémentaires                                                    | 3 h    |
| Total |                             |                                                                                             | 27 h   |

## Informations clés

|                     |                                                                                                                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PUBLIC              | Développeurs qui veulent construire des applications avec des grands modèles de langage                                                                                                                                |
| PRÉREQUIS           | Programmation Python (cours : Fondamentaux de la programmation Python), ou équivalent ; des bases de Docker aident (cours : Docker et Kubernetes). Aucune connaissance en machine learning n'est nécessaire            |
| DURÉE               | 27 h : 24 h de chapitres + 3 h de réserve                                                                                                                                                                              |
| FORMAT              | Cours courts suivis de TP guidés ; une même application reprise dans toutes les phases                                                                                                                                 |
| ENVIRONNEMENT DE TP | Un portable par participant (16 Go de RAM, Python 3, Docker, VS Code, Ollama pour les modèles ouverts) ; accès API à un LLM hébergé fourni par l'organisateur avec un plafond de dépenses ; aucun autre compte ni coût |
| ÉVALUATION          | Un TP validé à la fin de chaque phase et un projet final présenté au groupe ; attestation de formation pour les participants qui valident les deux                                                                     |
| RÉSULTAT            | Les participants savent construire une application d'agent avec RAG et outils, réduire son exposition à l'injection de prompt, mesurer sa qualité et son coût, et la déployer comme API conteneurisée                  |

## PHASE 01

6 H

# Fondamentaux des LLM

Les participants apprennent comment les LLM sont construits, de l'attention et des tokens au pré-entraînement, à l'ajustement par instructions et à l'optimisation par préférences, et ce que changent les modèles de raisonnement. Ils écrivent ensuite des prompts qui fonctionnent, obtiennent une sortie structurée, et appellent aussi bien des API hébergées que des modèles ouverts exécutés en local. TP validé : la même tâche d'extraction résolue avec un modèle hébergé et un modèle ouvert local, qui renvoient un JSON validé.

| #   | CHAPITRE               | CONTENU                                                                                                                                           | FORMAT          | HEURES |
|-----|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|--------|
| 1.1 | Transformers et LLM    | Attention, tokens, pré-entraînement, ajustement par instructions (instruction tuning), RLHF et DPO, modèles de raisonnement, panorama des modèles | Cours + atelier | 3 h    |
| 1.2 | Ingénierie des prompts | Schémas de prompts, prompts few-shot, sortie structurée                                                                                           | Cours + TP      | 1,5 h  |
| 1.3 | Utiliser les modèles   | API de LLM, sorties structurées, modèles ouverts (Ollama, Hugging Face)                                                                           | Cours + TP      | 1,5 h  |

# RAG et agents

Les participants ancrent les réponses dans leurs propres documents par la génération augmentée par la recherche, des embeddings et du découpage à la recherche hybride, au reclassement et à l'évaluation du RAG, puis laissent les modèles agir par l'usage d'outils et l'appel de fonctions, la planification et des frameworks multi-agents comme LangGraph et CrewAI. TP validé : un assistant de questions-réponses sur un ensemble de documents qui cite ses sources, étendu en agent qui appelle deux outils.

| #   | CHAPITRE                                    | CONTENU                                                                                        | FORMAT     | HEURES |
|-----|---------------------------------------------|------------------------------------------------------------------------------------------------|------------|--------|
| 2.1 | Génération augmentée par la recherche (RAG) | Embeddings, bases vectorielles, découpage, recherche hybride, reclassement, évaluation du RAG  | Cours + TP | 3 h    |
| 2.2 | Agents                                      | Usage d'outils, appel de fonctions, planification, frameworks multi-agents (LangGraph, CrewAI) | Cours + TP | 3 h    |

## PHASE 03

6 H

## MCP, sécurité et évaluation

Les participants exposent des outils par le Model Context Protocol, protègent l'application contre l'injection de prompt et les autres risques OWASP des applications LLM, puis mesurent la qualité avec des jeux d'évaluation et des LLM juges, tracent les appels et le coût, et servent l'application comme API dans un conteneur. TP validé : un serveur MCP utilisé par l'agent, une tentative d'injection bloquée par un garde-fou, et un rapport d'évaluation exécuté avant le déploiement.

| #   | CHAPITRE                  | CONTENU                                                                                     | FORMAT     | HEURES |
|-----|---------------------------|---------------------------------------------------------------------------------------------|------------|--------|
| 3.1 | MCP                       | Construire et connecter des serveurs et outils MCP                                          | Cours + TP | 1,5 h  |
| 3.2 | Garde-fous et sécurité    | Injection de prompt, OWASP Top 10 pour les applications LLM (2025), garde-fous              | Cours + TP | 1,5 h  |
| 3.3 | Évaluation et déploiement | Jeux d'évaluation, LLM juge, traçage, coût ; mise à disposition comme API dans un conteneur | Cours + TP | 3 h    |

## PHASE 04

6 H

## Projet final

Par équipes de deux ou trois, les participants construisent une application d'agent pour un cahier des charges écrit, en réutilisant les TP de phase, puis présentent sa qualité, sa sécurité et son coût comme ils le feraient devant un product owner.

| #   | CHAPITRE                          | CONTENU                                                                                                          | FORMAT | HEURES |
|-----|-----------------------------------|------------------------------------------------------------------------------------------------------------------|--------|--------|
| 4.1 | Cahier des charges et réalisation | Analyser le besoin, concevoir l'agent, construire le RAG sur les documents fournis, les outils et un serveur MCP | Projet | 3 h    |
| 4.2 | Évaluation et sécurisation        | Jeu d'évaluation et rapport, garde-fous, traçage et coût par requête, API conteneurisée                          | Projet | 1,5 h  |
| 4.3 | Démonstration et revue            | Démonstration en direct incluant une tentative d'attaque ; questions et retours                                  | Projet | 1,5 h  |

## PROJET FINAL

### Une application d'agent déployée

Un agent qui répond à partir d'un ensemble de documents en citant ses sources, agit par des outils et un serveur MCP, est testé contre l'injection de prompt, est mesuré sur un jeu d'évaluation avec son coût par requête, et tourne comme API conteneurisée.

**OUTILS** Python, une API de LLM et un modèle ouvert (Ollama), embeddings et base vectorielle, LangGraph, MCP, une bibliothèque de garde-fous (par ex. NeMo Guardrails), Ragas, Langfuse, FastAPI, Docker

**LIVRABLE** Dépôt avec code et jeu d'évaluation, rapport d'évaluation, démonstration en direct